

一，伙伴算法

它要解决的问题是频繁地请求和释放不同大小的一组连续页框，必然导致在已分配页框的块内分散了许多小块的空闲页面，由此带来的问题是，即使有足够的空闲页框可以满足请求，但要分配一个大块的连续页框可能无法满足请求。

伙伴算法（Buddy system）把所有的空闲页框分为11个块链表，每块链表中分布包含特定的连续页框地址空间，比如第0个块链表包含大小为 2^0 个连续的页框，第1个块链表中，每个链表元素包含2个页框大小的连续地址空间，....，第10个块链表中，每个链表元素代表4M的连续地址空间。每个链表中元素的个数在系统初始化时决定，在执行过程中，动态变化。

伙伴算法每次只能分配2的幂次页的空间，比如一次分配1页，2页，4页，8页，...，1024页(2^{10})等等，每页大小一般为4K，因此，伙伴算法最多一次能够分配4M的内存空间。

2，核心概念和数据结构

伙伴内存块：两个内存块，大小相同，地址连续，同属于一个大块区域。（第0块和第1块是伙伴，第2块和第3块是伙伴，但第1块和第2块不是伙伴）

伙伴位图：用一位描述伙伴块的状态位码，称之为伙伴位码。比如，bit0为第0块和第1块的伙伴位码，如果bit0为1，表示这两块至少有一块已经分配出去，如果bit0为0，说明两块都空闲，还没分配。

3，申请和回收过程

比如，我要分配4(2^2)页（16k）的内存空间，算法会先从free_area[2]中查看nr_free是否为空，如果有空闲块，则从中分配，如果没有空闲块，就从它的上一级free_area[3]（每块32K）中分配出16K，并将多余的内存（16K）加入到free_area[2]中去。如果free_area[3]也没有空闲，则从更上一级申请空间，依次递推，直到free_area[max_order]，如果顶级都没有空间，那么就报告分配失败。

释放是申请的逆过程，当释放一个内存块时，先在其对应的free_area链表中查找是否有伙伴存在，如果没有伙伴块，直接将释放的块插入链表头。如果有或板块的存在，则将其从链表摘下，合并成一个大块，然后继续查找合并后的块在更大一级链表中是否有伙伴的存在，直至不能合并或者已经合并至最大块 2^{10} 为止。

内核试图将大小为b的一对空闲块（一个是现有空闲链表上的，一个是待回收的），合并为一个大小为2b的单独块，如果它成功合并所释放的块，它会试图合并2b大小的块，

内核使用_rmqueue()函数来在管理区中找到一个空闲块，成功返回第一个被分配页框的页描述符，失败返回NULL。

4，伙伴算法的优缺点

优点：

较好的解决外部碎片问题

当需要分配若干个内存页面时，用于DMA的内存页面必须连续，伙伴算法很好的满足了这个要求

只要请求的块不超过512个页面(2K)，内核就尽量分配连续的页面。

针对大内存分配设计。

缺点：

1. 合并的要求太过严格，只能是满足伙伴关系的块才能合并，比如第1块和第2块就不能合并。
2. 碎片问题：一个连续的内存中仅仅一个页面被占用，导致整块内存区都不具备合并的条件
3. 浪费问题：伙伴算法只能分配2的幂次方内存区，当需要8K（2页）时，好说，当需要9K时，那就需要分配16K（4页）的内存空间，但是实际只用到9K空间，多余的7K空间就被浪费掉。
4. 算法的效率问题：伙伴算法涉及了较多的计算还有链表和位图的操作，开销还是比较大的，如果每次 2^n 大小的伙伴块就会合并到 $2^{(n+1)}$ 的链表队列中，那么 2^n 大小链表中的块就会因为合并操作而减少，但系统随后立即有可能又有对该大小块的需求，为此必须再从 $2^{(n+1)}$ 大小的链表中拆分，这样的合并又立即拆分的过程是无效率的。

Linux针对大内存的物理地址分配，采用伙伴算法，如果是针对小于一个page的内存，频繁的分配和释放，采用如slab和kmem_cache等方案

二，slab分配机制

slab分配器是基于对象进行管理的，所谓的对象就是内核中的数据结构（例如：task_struct,file_struct等）。相同类型的对象归为一类，每当要申请这样一个对象时，slab分配器就从一个slab列表中分配一个这样大小的单元出去，而当要释放时，将其重新保存在该列表中，而不是直接返回给伙伴系统，从而避免内部碎片。slab分配器并不丢弃已经分配的对象，而是释放并把它们保存在内存中。slab分配对象时，会使用最近释放的对象的内存块，因此其驻留在cpu高速缓存中的概率会大大提高。

三、内存对齐

对齐规则

- 1，结构体的成员变量，第一个成员变量的偏移量为 0。往后的每个成员变量的对齐值必须为编译器默认对齐长度（`#pragma pack(n)`）或当前成员变量类型的长度（`unsafe.Sizeof`），取最小值作为当前类型的对齐值。其偏移量必须为对齐值的整数倍
- *2，结构体本身，对齐值必须为编译器默认对齐长度（`#pragma pack(n)`）或结构体的所有成员变量类型中的最大长度，取最大数的最小整数倍作为对齐值

结合以上两点，可得知若编译器默认对齐长度（`#pragma pack(n)`）超过结构体内成员变量的类型最大长度时，默认对齐长度是没有任何意义的

```

1  type Part1 struct {
2      a bool
3      b int32
4      c int8
5      d int64
6      e byte
7  }
8  // 在开始之前, 希望你计算一下 Part1 共占用的大小是多少呢?
9  func main() {
10     fmt.Printf("bool size: %d\n", unsafe.Sizeof(bool(true)))
11     fmt.Printf("int32 size: %d\n", unsafe.Sizeof(int32(0)))
12     fmt.Printf("int8 size: %d\n", unsafe.Sizeof(int8(0)))
13     fmt.Printf("int64 size: %d\n", unsafe.Sizeof(int64(0)))
14     fmt.Printf("byte size: %d\n", unsafe.Sizeof(byte(0)))
15     fmt.Printf("string size: %d\n", unsafe.Sizeof("EDDYCJY"))
16 }
17 输出结果:
18 bool size: 1
19 int32 size: 4
20 int8 size: 1
21 int64 size: 8
22 byte size: 1
23 string size: 16
24
25 func main() {
26     part1 := Part1{}
27
28     fmt.Printf("part1 size: %d, align: %d\n", unsafe.Sizeof(part1),
29         unsafe.Alignof(part1))
30 }
31 输出结果:
32 part1 size: 32, align: 8

```

可以看到这个结构体占32个byte, 对齐宽度是8

CPU 读取内存是一块一块读取的, 块的大小可以为 2、4、6、8、16 字节等大小。块大小我们称其为内存访问粒度 (一般是4)

所以上述结构体占用内存如下：

```
1  axxx|bbbb|cxxx|xxxx|dddd|dddd|e...
```

之后再做一步整体对齐，在每个成员变量进行对齐后，根据规则 2，整个结构体本身也要进行字节对齐，因为可发现它可能并不是 2^n ，不是偶数倍。显然不符合对齐的规则，根据规则 2，可得出对齐值为 8。现在的偏移量为 25，不是 8 的整倍数。因此确定偏移量为 32。对结构体进行对齐

```
1  axxx|bbbb|cxxx|xxxx|dddd|dddd|exxx|xxxx
```